

BIG DATA

Application-Focused Exam Revision

COMPSCI4064 / COMPSCI5088 — Dr. Richard McCreadie

Based on 2020, 2023 (H/M/Practice), 2025 Exams with Full Solutions

How This Exam Works

EXAM FORMAT 2026

60 marks total | 1 hour 30 minutes | Closed-book | Crib sheet provided (Spark functions only)

Q1: Theory (Weeks 1-2) — Big Data dimensions, vertical vs horizontal scaling, parallelism challenges

Q2: Distributed Compute — MapReduce design, Hadoop overhead, Spark advantages, YARN

Q3: Distributed Data — HDFS, LSM trees, HBase/Cassandra, consistent hashing, Kubernetes
Week 10 content (legal/ethical) gets mixed into other questions

THE GOLDEN RULE: CONCEPT + CONTEXT = MARKS

Almost every question awards marks in pairs: 1 mark for the concept, 1 mark for linking it to the scenario.

Example: 'Mainframes are a single point of failure [1 mark], meaning if it breaks the satellite processing stops entirely [1 mark].'

If you write only the concept without linking to the scenario, you get HALF marks at best.

If you write only about the scenario without naming the concept, you get ZERO marks.

Question Types — How to Read Them

Keyword	What They Want	How to Structure
Discuss / Describe	Short paragraphs (marks / 2 = number of sentences)	Write flowing prose, weave concept + scenario together
List / X points expected	Bullet points, one per expected point	Bullet format: [Concept]: [Why it matters for this scenario]
True/False + explain	State T/F then justify with reasoning	T/F statement → 1-2 sentences of reasoning → link to scenario
MCQ + explain	Choose answer (0 marks!) then explain WHY	The MCQ choice itself is worth 0 marks. ALL marks are in the explanation

WARNING

MCQ choices are worth 0 marks. If you pick the right answer but don't explain it, you score nothing. The explanation IS the question.

Question 1: Theory (Weeks 1-2)

This question always gives you a scenario involving a company or system, then asks you to analyze it using Big Data concepts from weeks 1-2.

1A. Big Data Dimensions — Applied

You will be given a scenario and asked: which dimensions make this a Big Data problem and why?

EXAM QUESTION PATTERN

Asked in 2023 (H/M/Practice) and 2025. Every single year. Always scenario-based. Usually 6 marks = 3 dimensions x 2 marks each.

The 6 Dimensions

Dimension	What It Means	Scenario Trigger Words
Volume	Large amounts of data	TB/PB, millions of records, high-res images, large files
Velocity	Speed requirements for processing	Real-time, alerting, live, latency, within X seconds
Variety	Multiple data types/formats	Video + audio + text, CSV + custom formats, structured + unstructured
Veracity	Data quality / errors	Poor quality, sensor errors, data loss, cleaning needed
Variability	Workload changes over time	Peak periods, seasonal, scales up/down, 20% active but may increase
Complexity	Hard to process / multi-stage	Multiple processing stages, dependencies between data

HOW TO WRITE A DIMENSION ANSWER (2 marks each)

Pattern: [Dimension Name] [1 mark]: [Calculate/quote numbers from scenario] + [why this makes it a Big Data problem] [1 mark]

GOOD example (2023 satellite): 'Volume [1 mark]: Up to 500MB of images arriving each second (100 satellites x 20% active x 10MB each) constitutes a large volume to process [1 mark].'

BAD example: 'Volume: There is a lot of data.' ← No calculation, no link to scenario. 0-1 marks.

Always DO THE MATH. If the scenario says 100 satellites, 10MB each, 20% active → calculate 200MB/s normal, 500MB/s peak.

EXAM TIP

Not every dimension applies! In 2025, the answer was 'NO, this is NOT a Big Data problem' because the data was too small. Read the scenario carefully before claiming dimensions.

1B. Vertical vs Horizontal Scaling

Usually follows the dimensions question. A company is offered a single powerful machine (mainframe). Explain why that's bad.

EXAM QUESTION PATTERN

Asked in 2023 (H/M/Practice). 8 marks = 4 points x 2 marks. Each point: why mainframe is bad [1] + why it matters for THIS scenario [1].

4 STANDARD POINTS (memorize these)

1. Scaling limit: Mainframes have a max capacity for compute and storage [1 mark] — the company plans to double operations, so it will outgrow the mainframe [1 mark].
2. Cost: Mainframes are more expensive than commodity hardware clusters [1 mark] — for processing satellite images, many cheap machines are more cost-effective [1 mark].
3. Single point of failure: If the mainframe fails, ALL processing stops [1 mark] — for disaster alerting, downtime is unacceptable [1 mark].
4. Future-proofing: Mainframe cannot scale incrementally [1 mark] — horizontal scaling lets you add nodes as the satellite fleet grows [1 mark].

1C. Parallelism Challenges

Given a scenario with a multi-stage pipeline, discuss the challenges of building it in a distributed way.

EXAM QUESTION PATTERN

Asked in 2023 (H/M/Practice). 8 marks = 4 challenges x 2 marks. Each: name the concept [1] + link to scenario [1].

Challenge Concept	What to Write for Link
Data Splitting & Load Balancing	The [specific data] needs to be divided equally across machines
Communication & Network Overhead	The [intermediate data] must transfer between stages over the network
Data Duplication / Explosion	Each [item] produces X outputs, multiplying data volume
Fault Tolerance	With many machines, failures are likely; [specific data] must not be lost
Synchronization	Stage N cannot start until Stage N-1 finishes for each [data item]
Heterogeneous Hardware	Different machines may process at different speeds, causing stragglers

1D. Batch vs Streaming

Asked in 2023 Practice and 2025. You must pick one AND argue why it's better for the given scenario.

EXAM QUESTION PATTERN

'Discuss which is better and why.' 4 marks = state your choice + 2 points x 2 marks each. You MUST contrast against the alternative.

Argue FOR Batch When...	Argue FOR Streaming When...
Data needs to be complete before processing	Low latency / real-time results are needed
Processing can wait (end-of-day reports)	Data arrives continuously and must be acted on
Parallelizable by chunks (video segments)	Avoids storing large raw data (process and discard)

Error handling is simpler with complete data

Pipeline stages can overlap (CPU + GPU utilization)

TEMPLATE: BATCH VS STREAMING ANSWER

[Streaming/Batch] is the better solution because:

1. [Advantage] [1 mark]: [Why this matters more than the alternative for this scenario] [1 mark].
2. [Advantage] [1 mark]: [Contrast — 'unlike batch processing which would...' or 'whereas streaming would...'] [1 mark].

Example (2025 WhiskyAI): 'Streaming is better because: (1) Stream processing defines computation as a pipeline where items at different stages can be processed in parallel [1 mark]. This means the next batch can be prepared while the GPU learns from the current batch [1 mark]. (2) This avoids the idle-resource problem [1 mark], whereas batch processing forces CPU and GPU to take turns [1 mark].'

1E. Legal and Ethical Implications

Mixed into Q1 or other questions. Given a scenario where data is collected, discuss legal/ethical concerns.

EXAM QUESTION PATTERN

Asked in 2025. 6 marks = 3 points x 2 marks. Straightforward but you **MUST** name specific legal concepts.

Concept	When to Use It
Privacy	Video/images capturing people's faces, location tracking, personal data
Informed Consent	Data subjects didn't agree to their data being used this way
Data Retention (GDPR/DPA)	EU law limits how long data can be stored — affects model retraining
Purpose Limitation	Data collected for X being used for Y without permission
De-identification	Raw data contains identifiable info (faces, plates) that must be anonymized
Right to Erasure	Individuals can request their data be deleted — problematic for trained models

Question 2: Distributed Compute

This question covers MapReduce design, Hadoop overhead costs, Spark advantages, and YARN. It's always scenario-based — you're given a real-world application and asked to design/analyze solutions.

2A. Designing a MapReduce Solution

You're given a multi-stage processing task and must map it to MapReduce operations. This is the hardest question type — you must think about what the mapper does, what the key is, and how the reducer groups things.

EXAM QUESTION PATTERN

Asked in 2020 and 2023 (H/M/Practice). 5-14 marks. You must specify inputs, outputs, and key choices.

THE MAPREDUCE DESIGN FORMULA

Step 1: Identify what needs to be GROUPED — that becomes your KEY.

Step 2: Identify what can be done per-record independently — that's your MAP.

Step 3: Identify what needs all records for a group — that's your REDUCE.

For each operation, state: Operation type [1 mark] + What it does [1 mark] + Input <key, value> [1 mark] + Output <key, value> [1 mark]

TEMPLATE:

Map: [Does X per record]. Input: <[default key], [record]>. Output: <[grouping key], [relevant data]>.

Reduce: [Aggregates/combines all values for each key]. Input: <[key], List<[data]>>. Output: [final result].

Then add WHY paragraph: 'We chose [operation] as [map/reduce] because [it can/cannot be done per-record] [1 mark].'

Real Exam: News Summarizer (2023)

SCENARIO

4 stages: content extraction → topic identification → paragraph grouping → summarization.

Question: Implement in MapReduce. State operation types, inputs/outputs. Then explain WHY.

FULL MARKS ANSWER

Map: Content extraction + topic identification [1 mark]. Input: <articleid, article> [1 mark]. Output: <topic, paragraph> [1 mark].

Reduce: Performs summarization [1 mark]. Input: <topic, List<paragraph>> [1 mark]. Output: summary [1 mark].

Why: Content extraction and topic identification can be executed per-article, so they aggregate into a single Map step [1 mark]. Setting output key = topic means the reducer automatically groups paragraphs by topic [1 mark]. Summarization requires all paragraphs on a topic, making it a reducer [1 mark].

Real Exam: SELECT DISTINCT (2020)

SCENARIO

Text file, each line has two integers (weight, volume). Implement SELECT DISTINCT Weight, Volume.

FULL MARKS ANSWER

Map: For each line, parse the two integers. Emit key = binary concatenation of both integers, value = null [3 marks].

Reduce: Output the key regardless of values (since MapReduce groups by key, duplicates are already merged) [2 marks].

KEY INSIGHT: MapReduce's shuffle-and-sort phase groups identical keys together. By making the (weight, volume) pair the key and emitting null values, the DISTINCT operation happens automatically during grouping.

EXAM TIP

The combiner follow-up: If asked to add a combiner, it acts as a 'mini reducer' — output one key-value pair per unique key with null value. Often NO amendments needed to map/reduce functions.

2B. Hadoop Overhead Costs

After you design the MapReduce solution, they ask: what are the overhead costs? This is NOT about the cost of your operations — it's about Hadoop's inherent inefficiencies.

EXAM QUESTION PATTERN

Asked in 2023 (H/M) and indirectly in 2025. 6-8 marks = 3-4 points x 2 marks.

CRITICAL: They say 'We do NOT expect you to discuss the cost of the operations themselves... but rather aspects of Hadoop that will result in efficiency overheads.'

THE 4 HADOOP OVERHEADS (memorize all 4)

1. Small file problem: Data loading will be slow [1 mark] because HDFS is not optimized for many small files (NameNode metadata overhead) [1 mark].
2. Intermediate sort cost: There will be overhead performing the intermediate sort [1 mark] of map output data based on keys [1 mark].
3. Network transfer (shuffle): Significant network latency [1 mark] as map outputs are transferred across the network to be grouped by key [1 mark].
4. Disk write cost: Data is persisted to disk after the map sort [1 mark], adding I/O overhead [1 mark].

ALWAYS link to scenario: 'Loading will be slow because each news article is stored in a separate file and HDFS does not like small files.'

WARNING

For 2025's question about 'why is this difficult in Hadoop' — the pattern shifts slightly: they want Hadoop LIMITATIONS (not just overheads). Add: (1) keyless transformation problem [1 mark] — Map and Reduce both require keys [1 mark]. (2) High job startup costs [1 mark] — makes frequent computation (every 10 seconds) impractical [1 mark].

2C. Spark Advantages Over Hadoop

After discussing Hadoop's problems, the follow-up is always: why would Spark be better?

EXAM QUESTION PATTERN

Asked in 2023 (H/M/Practice) and implicitly in 2025. 6-8 marks = 3-4 points x 2 marks.

THE 4 SPARK ADVANTAGES (pick the ones relevant to the scenario)

1. No HDFS dependency: Spark does not rely on HDFS for data [1 mark], so initial read costs are lower / can use local storage [1 mark].
2. In-memory processing: Spark does not write intermediate data to disk [1 mark], avoiding the I/O overhead of Hadoop's shuffle [1 mark].
3. Better serialization: Transformation overheads are generally lower [1 mark] due to more efficient serialization [1 mark].
4. Flexible programming model: Operations defined as separate transformations [1 mark] rather than rigid Map/Reduce structure [1 mark].

INVALID answers (will score 0): chaining maps/reduces, streaming support, language support (unless scenario specifically involves these).

EXAM TIP

In 2023 Practice (mobile phones scenario), they asked WHY Spark over Hadoop for phones. The answers are different: (1) No HDFS — mobile data upload costs too high. (2) In-memory — less battery drain. (3) Faster tasks — less local compute. (4) More languages — wider customer base. ALWAYS adapt to scenario.

2D. Spark Stages and Dependencies

MCQ + explain format. Remember: the MCQ choice is worth 0 marks.

EXAM QUESTION PATTERN

Asked in 2020. 9 marks: 0 for MCQ + 3 for explaining stages + 6 for performance reasoning.

Dependency	Definition	What Happens
Narrow	Each parent partition used by AT MOST one child partition	Grouped and pipelined within a stage — no network shuffle
Wide	Each parent partition used by MULTIPLE child partitions	Requires shuffle — marks stage boundaries

FULL 9-MARK ANSWER TEMPLATE (2020 style)

Answer: (a) — Narrow = parent partition used by at most one child; Wide = used by multiple children. [0 marks for choice]

Stage creation [3 marks]: Narrow operations are grouped and pipelined in a stage. Tasks on partitions with narrow dependencies are scheduled on the executor holding the parent partition. Wide operations require data shuffled across nodes and mark stage boundaries.

Performance reasoning [6 marks]: This replicates what a seasoned MapReduce programmer would do by hand — identify operations needing only local data and execute them in a pipelined fashion [2 marks]. If producing output for partition Z only needs computation on one partition of Y, which only needs one partition of X, all three computations happen in-memory sequentially on the same

executor [3 marks]. This gives significant network overhead reduction for negligible memory overhead [1 mark].

2E. YARN and Containers

YARN questions test whether you understand what it replaced and how modern containers differ from YARN containers.

EXAM QUESTION PATTERN

Asked in 2023 (M/Practice). 3-8 marks. Ranges from 'why is YARN better' to 'compare YARN containers vs Docker containers'.

Why YARN is Better Than MR1

STANDARD 3-MARK ANSWER

1. Better resource utilization [1 mark] — handles variable-size resource requests instead of fixed task slots [1 mark].
2. Multi-application support [1 mark] — enables Spark, HBase etc. to run concurrently in the same Hadoop ecosystem.

YARN Containers vs Docker/Kubernetes Containers

Aspect	YARN Container	Docker/K8s Container
What it is	A unit of work (OS process, usually JVM)	Code + OS context + libraries bundled together
Execution	Run by Node Manager process	Run by OS-level virtualization engine (Docker/K8s)
Scope	Hadoop ecosystem only	Any application
Isolation	Shares host OS environment	Sandboxed with own drivers and libraries
Visibility	Can monitor entire host machine	Can only see allocated resources

2F. MapReduce Execution Steps (Full Pipeline)

In 2023 Practice, they asked for 7 steps of how a video is processed through MapReduce. This tests deep understanding of the full pipeline.

EXAM QUESTION PATTERN

Asked in 2023 Practice. 14 marks = 7 steps x 2 marks (name step + describe what happens).

THE 7 STEPS (applied to video encoding scenario)

1. Read (HDFS → Map): The video is read from HDFS as a series of blocks [1 mark], where each block = a batch of image frames [1 mark].
2. Map: The encoder executes per frame [1 mark], downsampling each frame to lower resolution [1 mark].
3. Sort: Encoded images are sorted by batch [1 mark] using the key-frame identifier as key [1 mark].
4. Write (Map → Disk): Ordered frames are written [1 mark] to local HDFS data node [1 mark].
5. Read (Disk → Reduce): Ordered frames are read by key [1 mark], blocks copied to reducer machine and merged [1 mark].

6. Reduce: Reads all frames for a key-frame and builds the key-frame block [1 mark], including compression [1 mark].
7. Write (Reduce → HDFS): Key-frame blocks written to local HDFS [1 mark] as reducer output, then replicated [1 mark].

EXAM TIP

The generic pattern is always: Read → Map → Sort → Write → Shuffle/Read → Reduce → Write. But you **MUST** describe each step in terms of the SPECIFIC scenario (frames, blocks, key-frames, etc.), not generic terms.

2G. Can Reducers Start Before Mappers Finish?

Asked in 2020. A tricky synthesis question.

COMPLETE ANSWER (3 marks)

Reducers cannot start **PROCESSING** before all mappers finish [1 mark] because each reducer needs to merge output from potentially all mappers.

However, reducers **CAN** start **TRANSFERRING** data before all mappers finish (slowstart mechanism) [1 mark].

This overlaps CPU-intensive map processing with IO-intensive shuffle transfers, improving overall utilization [1 mark].

2H. Stateful Distributed Processing

New in 2025. Given a scenario with periodic synchronization (like ML training), explain why it's hard.

EXAM QUESTION PATTERN

Asked in 2025. 6 marks: 2 for identifying it as 'stateful' + 4 for reasons why it's difficult (2 points x 2 marks).

ANSWER TEMPLATE

This training process requires global state — the application is stateful [2 marks].

Implementing stateful distributed processing is difficult because:

1. Synchronization adds an intermediate step where cluster machines are waiting idle [2 marks].
2. Each task stores additional state (neural network weights), which over time risks out-of-memory errors [2 marks].
3. (Alt) Network transfer of state adds cost [2 marks].
4. (Alt) Error recovery is complex — a worker failure loses associated state, requiring re-execution of entire batch [2 marks].

Question 3: Distributed Data

Covers HDFS, LSM trees, HBase/BigTable, Cassandra, consistent hashing, and Kubernetes. Always scenario-based.

3A. HDFS — Applied

HDFS Read Process & Design Implications

The classic MCQ + explain. They give you an MCQ about how clients read from HDFS, then ask you to explain the design implications.

EXAM QUESTION PATTERN

Asked in 2020. 8 marks: 0 for MCQ + 5 for explanation + 3 for wrong options.

COMPLETE ANSWER

Answer: (b) Client receives block locations from NameNode, then reads directly from DataNodes.

Why this design [3 marks]: Clients contact NameNode ONLY for block locations and cache them. All I/O then goes directly to DataNodes. This means the NameNode is removed from the data transfer path, allowing the system to scale despite having a single NameNode.

Requirement [2 marks]: The NameNode must maintain very low latency, which requires all filesystem metadata to fit in RAM. This imposes either high RAM requirements on NameNode hardware or an upper limit on filesystem size.

Why others are wrong: (a)/(c) would make NameNode a bottleneck [1 mark]. (d) would require all DataNodes to know all block locations — enormous network overhead for block creation/migration [1 mark], plus same RAM requirements as NameNode on every DataNode [1 mark].

NameNode vs DataNode Hardware

EXAM QUESTION PATTERN

Asked in 2020. 6 marks. MCQ + explain.

ANSWER

Answer: (c) NameNode needs higher RAM than DataNode.

NameNode stores complete filesystem image in RAM [1 mark]. Disk is less important for NameNode — typical disk > typical RAM [2 marks].

DataNodes need moderate RAM for caching [1 mark] but high disk capacity [1 mark] as they store the bulk of the data [1 mark].

HDFS vs NFS/RAID — Comparison

EXAM QUESTION PATTERN

Asked in 2023 (H/M). 8 marks = 4 points x 2 marks. Always paired: what's different [1] + impact [1].

Point	What (1 mark)	Impact (1 mark)
Replication	HDFS replicates on-site across nodes	Better throughput from more disk/network capacity

Fault tolerance	Cluster of machines vs single NFS server	Protects against single-point hardware failure
Complexity	HDFS is more complex to manage	Needs orchestration, multiple data nodes
Cost	HDFS requires redundant hardware	More expensive overall
Storage policies	HDFS supports intelligent device selection	NFS has no equivalent
File size optimization	HDFS optimized for large sequential files	Audio logs faster, sensor data (small files) slower

Replication Factor Effects

EXAM QUESTION PATTERN

Asked in 2023 Practice. Three sub-questions about increasing replication factor.

REPLICATION FACTOR ANSWERS

Write throughput: HDFS uses pipelined replication [2 marks]. N replicas cost $\sim N \times \text{network_latency}$ on top of first write, which is negligible on 1Gbps+ links [2 marks]. Write throughput only slightly reduced [1 mark].

Read latency: HDFS reads from the closest replica [2 marks]. More replicas = more locations = higher chance of a nearby replica, so latency potentially decreases [2 marks].

Resource usage: More disk usage across DataNodes (more copies) [2 marks]. Higher NameNode memory (must store locations of additional replicas) [2 marks]. More bandwidth for block reports from DN to NN [2 marks].

3B. LSM Trees — Applied

True/False questions about LSM tree properties. The key is that the True/False itself doesn't matter — marks are for the EXPLANATION.

EXAM QUESTION PATTERN

Asked in 2020 and 2025. 8 marks in 2020 (4 T/F x 2 marks), 7 marks in 2025. They explicitly say: 'if an answer gives the wrong T/F but provides correct arguments, it won't be penalized.'

Statement	T/F	Explanation
Lower read latency as more items added	FALSE	More SSTable files to scan = higher read latency. Caching helps for skewed workloads but doesn't offset in general case.
High write throughput	TRUE	In-memory buffering (MemTable) + write-ahead logging (append-only) = fast writes. Only sequential disk I/O.
Compaction reduces retrievable items	FALSE	Compaction only removes non-visible data (old versions, tombstones). Retrievable items unchanged.
Only works with master-workers architecture	FALSE	LSM is per-node design. Both HBase (master-workers) AND Cassandra (peer-to-peer) use it.

LSM Write Steps (HBase Region Server)

EXAM QUESTION PATTERN

Asked in 2025 (MoonShuttle). 7 marks: 1 for naming LSM + 4 for write steps + 2 for fast lookup explanation.

ANSWER

Storage mechanism: Log-Structured Merge (LSM) Tree [1 mark].

Write steps:

1. Write trade into Write Ahead Log (append to file) [1 mark].
2. Store trade in MemTable (in-memory sorted structure) [1 mark].
3. Periodically sort, compress, and flush to SSTable on disk [1 mark].
4. SSTables compacted periodically to save space [1 mark].

Fast lookup: Recent trades are in the MemTable [1 mark], which acts as a cache accessible at main-memory speeds [1 mark].

Bloom Filters

EXAM QUESTION PATTERN

Asked in 2025 (MoonShuttle follow-up). 3 marks.

ANSWER

Statement that fewer large files reduce index lookup time is FALSE [1 mark].

HBase uses Bloom filters for SSTable file lookup [1 mark].

Bloom filters have $O(k)$ complexity (k = number of hash functions), which can run in parallel = constant time lookup [1 mark].

3C. HBase vs Cassandra — Applied

Is HBase a Row Store or Column Store?

EXAM QUESTION PATTERN

Asked in 2020. 4 marks. Synthesis question.

ANSWER

HBase is essentially a ROW store, although called a 'fat column store' [1 mark].

Cells sorted first by rowkey, then by column identifier = row-major format [1 mark].

Columns grouped into column families, each stored in separate files — so queries accessing one CF don't touch others [1 mark].

For it to be a true column store, each column would need its own CF, or queries would only ever access one CF [1 mark].

Cassandra vs HBase — When to Use Which

EXAM QUESTION PATTERN

Asked in 2025 (MoonShuttle). 4 marks = 2 points x 2 marks.

ANSWER

1. No central master index: Cassandra has no central scan functionality [1 mark], making it hard for governments to get a full customer list [1 mark].

2. Geographic partitioning: Cassandra's hashing scheme selects which node stores data [1 mark], enabling geographic partitioning by country while maintaining searchability [1 mark].

WARNING

'HBase is not a file store — it's a column store and cannot store student files.' This was actually a trap question in 2023 Practice to catch students using AI. If the question asks whether HBase can replace file storage, the answer is simply NO.

3D. Consistent Hashing — Applied

They test whether you truly understand consistent hashing by giving True/False statements with tricky scenarios.

EXAM QUESTION PATTERN

Asked in 2020 and 2023 (H/M). 6-8 marks. True/False + explain format.

With Order-Preserving Hash Function

Statement	T/F	Why
Item always hashed to same node regardless of node changes	FALSE	If a node is added/removed in the range containing the item, placement changes. But changes outside that range don't affect it. Give a concrete example with nodes A, B and item x.
Storage load balanced across nodes	FALSE	Order-preserving hash preserves the distribution of item values. If values aren't uniform, load won't be balanced.
Range queries perform well	TRUE	This is the entire purpose of order-preserving hash — adjacent keys map to adjacent positions.
Point queries better than random hash	FALSE	Point queries only access one node regardless of hash function. Order-preserving may actually be WORSE due to load imbalance.

How Consistent + Order-Preserving Combine

STANDARD ANSWER (6 marks in 2023)

Consistent hashing partitions the ID space across nodes [1 mark] and assigns ID X to the first node with ID $\geq X$ [1 mark].

Order-preserving hash maps input key Y to an ID A such that key Z > Y maps to ID B > A [1 mark].

Combined: input key $\rightarrow$ order-preserving hash $\rightarrow$ position on ring $\rightarrow$ consistent hashing $\rightarrow$ assigned to node [1 mark].

This enables range queries (adjacent keys go to adjacent/same nodes) while maintaining the scalability of consistent hashing [2 marks].

3E. Quorum and Consistency

A tricky 2020 question that most students got wrong.

EXAM QUESTION PATTERN

Asked in 2020. 6 marks. MCQ with a trap.

THE TRAP

You'd expect $r + w > N$ to guarantee strong consistency. But the correct answer is (d) NONE OF THE ABOVE.

Why? Quorum assumes writes always succeed — all w replicas update fully before any read. But without 2PC, Paxos, or Raft to guarantee atomic writes, a partial write failure means $r + w > N$ doesn't guarantee you'll read the latest value.

KEY INSIGHT: Quorum provides strong consistency ONLY with additional atomic write protocols.

3F. Distributed vs Decentralized

EXAM QUESTION PATTERN

Asked in 2023 (H/M). 3 marks. Quick definition question.

ANSWER

Distributed: Storage spread across many physical machines, usually with data replication [1 mark]. However, most distributed solutions (e.g., HDFS) have a centralized management service/master [1 mark].

Decentralized: A subset of distributed solutions that LACK this central service/master (e.g., Cassandra) [1 mark].

3G. Kubernetes

EXAM QUESTION PATTERN

Asked in 2025 (MoonShuttle). 5 marks: 2 for definition + 3 for benefits.

ANSWER

Kubernetes is an open-source container orchestration system [1 mark] for automating software deployment, scaling, and management [1 mark].

Benefits:

1. Automate deployment of Cassandra nodes when packaged as containers [1 mark].
2. Automatically restart failed nodes [1 mark].
3. Abstracts away physical hardware, enabling unified deployment across heterogeneous regions [1 mark].

3H. Virtual Nodes (Cassandra)

EXAM QUESTION PATTERN

Asked in 2023 Practice. 5 marks.

ANSWER

Virtual nodes allow one physical node to appear as multiple nodes for data placement purposes [1 mark].

Primary purpose: better load balancing [1 mark].

With 50 nodes, probably NOT needed [1 mark] — 50 unique locations should provide good fault tolerance [1 mark].

Exception: if the hashing scheme causes data to cluster on a subset of nodes, virtual nodes would help [1 mark].

Appendix A: Spark Crib Sheet — Quick Reference

These functions are provided on the crib sheet. You don't need to memorize syntax, but you **MUST** know when to use each one.

Transformations (Lazy)

Function	Use When
map(func)	Transform each element 1-to-1
filter(func)	Keep only elements matching condition
flatMap(func)	Each element produces 0 or more outputs
distinct()	Remove duplicates
sample(frac)	Take a random sample
union(other)	Combine two RDDs
intersection(other)	Keep only elements in both RDDs
subtract(other)	Remove elements that appear in other RDD
cartesian(other)	All pairs from both RDDs

Key-Value Transformations

Function	Use When
reduceByKey(func)	Aggregate values per key — ALWAYS prefer over groupByKey
groupByKey()	Group values by key — AVOID (shuffles all data)
sortByKey()	Sort by key
join(other)	Combine two datasets on matching keys
mapValues(func)	Transform only values (preserves partitioning)
flatMapValues(func)	Each value produces 0+ outputs (preserves partitioning)
keys() / values()	Extract just keys or values
aggregateByKey(zero)(seq,comb)	Like reduceByKey but output type can differ from input
combineByKey(create,merge,combine)	Most general per-key aggregation

Actions (Trigger Execution)

Function	Use When
collect()	Get all results to driver — CAREFUL with large data
count()	Count elements
reduce(func)	Aggregate entire RDD to single value
first() / take(n)	Get first or first n elements
saveAsTextFile(path)	Write output to file system

CRITICAL: reduceByKey vs groupByKey

reduceByKey: Combines values **LOCALLY** on each partition first (like a combiner), then shuffles. Efficient.

groupByKey: Shuffles **ALL** data across the network, **THEN** groups. Very expensive.

ALWAYS use `reduceByKey` unless you genuinely need the full list of values.
If an exam question asks you to count/sum/aggregate, and you use `groupByKey`, expect to lose marks.

Solution Templates

For Spark coding questions, these patterns cover most scenarios:

Pattern	Template
Count by category	<code>input.map(r => (category, 1)).reduceByKey((a,b) => a+b)</code>
Filter + Aggregate	<code>input.filter(condition).map(r => (key, value)).reduceByKey(func)</code>
Join two datasets	<code>A.map(r => (sharedKey, valA)).join(B.map(r => (sharedKey, valB)))</code>
Top-K	<code>...reduceByKey(sum).map((k,v) => (v,k)).sortByKey(false).take(K)</code>
Deduplication	<code>input.map(r => compositeKey).distinct()</code>
Reverse graph edges	<code>edges.map((src,dst) => (dst,src))</code>

FINAL EXAM STRATEGY

1. Read the GUIDANCE first — it tells you exactly how many points to make and how marks are split.
2. Count your points: If guidance says '4 points expected, 2 marks each', write exactly 4 bullet points.
3. Every point = concept + scenario link. Never write a concept without connecting it to the given scenario.
4. Do the math: If numbers are in the scenario, calculate them (e.g., 100 satellites x 10MB x 20% = 200MB/s).
5. Use scenario-specific nouns: Don't say 'the data'. Say 'the satellite images' or 'the trade records'.
6. For MCQs: Spend 90% of your time on the explanation, not choosing the answer.
7. Time management: 60 marks in 90 minutes = 1.5 minutes per mark. A 6-mark question gets 9 minutes max.